GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Qué es un fichero

Todas las estructuras de datos que hemos visto hasta ahora utilizan memoria principal. Esto tiene dos limitaciones importantes:

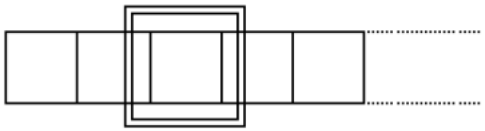
- Los datos desaparecen cuando el programa termina.
- La cantidad de los datos no puede ser muy grande debido a la limitación de la memoria principal.

Por eso existen también estructuras especiales que utilizan memoria secundaria: los ficheros. El fichero es además una estructura dinámica, en el sentido de que su tamaño puede variar durante la ejecución del programa dependiendo de la cantidad de datos que tenga.

Tipos de acceso

Al estar en memoria secundaria, no todos los elementos del fichero son accesibles de forma inmediata. Solamente se puede acceder cada vez a un único elemento del fichero, que se denomina puntero del fichero. Dependiendo de cómo se desplaza por el fichero, podemos distinguir dos tipos de acceso:

- **Acceso secuencial**: El puntero del fichero sólo puede moverse hacia delante a partir del primer elemento y siempre de uno en uno.
- **Acceso directo**: El puntero del fichero se puede situar directamente en cualquier posición del fichero. Es un acceso similar al utilizado en los Arrays unidimensionales.



GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Qué es un fichero

Ficheros binarios y ficheros de texto

Existen dos tipos principales de ficheros:

- **Ficheros de texto**: Contienen secuencias de caracteres separadas por saltos de línea (tipo str). El teclado (entrada estándar) y la pantalla (salida estándar) se consideran también ficheros de texto. Al leer o escribir variables de un fichero de texto se pueden realizar ciertas conversiones. Por ejemplo, cuando escribimos un entero con valor 10, este entero se convierte en los caracteres '1' y '0'.
- **Ficheros binarios**: Contienen secuencias de bytes. Los elementos se almacenan en el fichero exactamente igual que están almacenados en memoria principal, es decir, al leer o escribir no se realiza ningún tipo de conversión.

Modo de acceso a los ficheros

Cuando intentamos acceder a un fichero existen dos únicas acciones posibles: leer o escribir, según estas acciones se pueden definir en ficheros de entrada (abiertos para lectura), ficheros de salida (abiertos para escritura) o ficheros de lectura / escritura (abiertos para ambas operaciones).

Los ficheros residen en un sistema de archivos y tendremos que saber la ruta de acceso al mismo para poder utilizarlo, dependiendo del SSOO de base los separadores cambian: / o \, con lo que recomendamos utilizar las herramientas de los lenguajes o librerías asociadas para crear estas rutas de acceso, evitando trabajar con características específicas de los SSOO.

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Qué es un fichero

Procesamiento de un fichero

Siempre que queramos realizar cualquier operación con ficheros se debe seguir el siguiente esquema:

Apertura de Fichero → Operaciones → Cierre del fichero

Es importante seguir el esquema y sobre todo no olvidar el cierre del fichero, si no queremos perder datos. Los SSOO no escriben inmediatamente los datos de memoria a disco, utilizan un buffer intermedio por eficiencia, si no cerramos el fichero no nos aseguramos que este buffer se descargue a disco y podremos perder las últimas operaciones.

Qué son los flujos

Los archivos o ficheros almacenados se tienen que tratar por parte del SSOO y de Python. Para esa gestión se crea un objeto especial en memoria. Por tanto, un flujo no es más que un objeto de control que gestiona un fichero en disco, creado en el momento de la apertura y destruido cuando se cierra el archivo. El flujo representa al archivo dentro de nuestro programa y da acceso a sus datos.

Bajo Python no se habla de flujos, se asocia más a lenguajes como C++ o Java, se utiliza directamente fichero o archivo, pero este concepto también existe y hay paquetes que lo implementan: **io** (https://docs.python.org/es/3/library/io.html) que utilizaría conceptos similares al lenguaje C++ o Java.

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Acceso a ficheros de Texto y Binarios

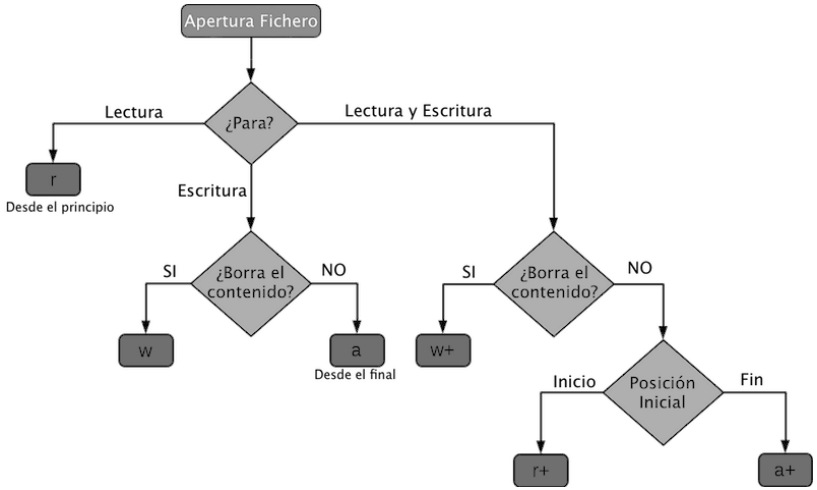
Apertura de ficheros

Python tiene una función **open()** incorporada para abrir un archivo. Esta función devuelve un objeto archivo, también llamado identificador, ya que se usa para leer o modificar el archivo en consecuencia (sería similar al concepto del lenguaje C).

```
f = open("test.txt") # open file in current directory
f = open("C:/Python38/README.txt") # specifying full path
```

Modos de acceso.

Podemos especificar el modo al abrir un archivo en el momento de su apertura. En el modo, especificamos si queremos leer **(r)**, escribir (**w)** o agregar (**a)** al archivo. También podemos especificar si queremos abrir el archivo en modo texto (**t**) o en modo binario (**b**).



GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Acceso a ficheros de Texto y Binarios

Modos de acceso.

El valor predeterminado es leer en modo texto (**rt**) . En este modo, obtenemos cadenas (**str**) al leer del archivo. Por otro lado, el modo binario devuelve bytes (**bytes**) y este es el modo que se debe utilizar cuando se trata de archivos que no son de texto, como imágenes o archivos ejecutables.

```
f = open("test.txt") # equivalent to 'r' or 'rt'
f = open("test.txt",'w') # write in text mode
f = open("img.bmp",'r+b') # read and write in binary mode
```

Python no utiliza en la lectura y escritura una codificación estricta, por lo que debemos especificarla en el momento de la apertura si no queremos tener problemas. De hecho, la codificación por defecto que se usará si no la especificamos es dependiente de la plataforma, así en Windows utilizará **cp1252** pero bajo Linux se utilizará **utf-8**.

Debemos especificar la codificación correcta o no se mostrarán los caracteres de forma adecuada, por lo que si el fichero no lo hemos creado nosotros habrá que encontrar la codificación y usarla. Cuando creemos nosotros los ficheros utilizaremos siempre que sea posible **utf-8** independientemente de la plataforma en la que se ejecute.

```
f = open("test.txt", mode='r', encoding='utf-8')
```

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Acceso a ficheros de Texto y Binarios

Cierre de ficheros

Cuando terminemos de realizar operaciones en el archivo, debemos cerrarlo correctamente. Cerrar un archivo liberará los recursos que estaban vinculados con el archivo. Se hace usando el método **close()** disponible en Python. Python tiene un recolector de basura para limpiar objetos sin referencia, pero no debemos confiar en él para cerrar el archivo.

```
f = open("test.txt", encoding = 'utf-8')
# realizar operaciones de lectura escritura
f.close()
```

Queda una cuestión: si en el programa se produce alguna excepción, el fichero no se cerrará correctamente y podemos crear malos funcionamientos, con lo que el protocolo que debemos usar para la gestión de archivos es el que mostramos a continuación.

```
try:
f = open("test.txt", encoding = 'utf-8')
# realizar operaciones de lectura escritura
finally:
f.close()
```

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Acceso a ficheros de Texto y Binarios

Uso de with con ficheros

Hemos vistos el protocolo estándar y los problemas que nos puede producir el no capturar correctamente las excepciones. Para evitar estas situaciones se ha implementado el uso de la palabra reservada **with**, que se encargará de cerrar adecuadamente el fichero y tratar las excepciones si ocurrieran.

```
with open("test.txt", encoding = 'utf-8') as f:
# realizar operaciones de lectura escritura
```

Escritura y lectura de información en ficheros

Python trabaja con los ficheros de forma similar al C, por lo que no les dota de una estructura y es difícil la actualización y manejo de bases de datos de ficheros.

Para escribir en un archivo en Python, necesitamos abrirlo en escritura **w**, agregar **a** o modo exclusivo **x** creación. Debemos tener cuidado con el modo **w**, ya que se sobrescribirá en el archivo si ya existe. Debido a esto, se borran todos los datos anteriores.

La escritura de una cadena o secuencia de bytes (para archivos binarios) se realiza mediante el método **write()**. Este método devuelve el número de caracteres escritos en el archivo.

Debemos incluir los caracteres de nueva línea nosotros mismos para distinguir las diferentes líneas.

```
with open("test.txt",'w',encoding = 'utf-8') as f:
f.write("my first file\n")
f.write("This file\n\n")
f.write("contains three lines\n”)
```

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Acceso a ficheros de Texto y Binarios

Para leer un archivo en Python, debemos abrir el archivo en modo (**r**) de lectura. Hay varios métodos disponibles para este propósito. Podemos usar el método de **read (tamaño)** para leer el número de tamaño de los datos. Si no se especifica el parámetro de tamaño, se lee y regresa hasta el final del archivo.

```
f = open("test.txt",'r',encoding = 'utf-8')
f.read(4) # read the first 4 data
f.read(4) # read the next 4 data
f.read() # read in the rest till end of file
f.read() # further reading returns empty sting
```

Una vez que se llega al final del archivo, obtenemos una cadena vacía en la lectura adicional.
Podemos cambiar el cursor de lectura usando la instrucción **seek(posicion)**. También podemos determinar la posición actual mediante **tell()**. Ambos métodos trabajan a nivel de bytes.

```
f.tell() # get the current file position
f.seek(0) # bring file cursor to initial position
```

Ejemplo de acceso

```
for line in f:
print(line, end = '')
```

En ficheros de texto podemos usar el método **readline()** para leer líneas individuales de un archivo. Este método lee un archivo hasta el salto de línea, incluido el carácter de salto de línea. O podemos usar el método **readlines()** que devuelve una lista con las líneas restantes de todo el archivo. Todos estos métodos de lectura devuelven valores vacíos cuando se alcanza el final del archivo (EOF).

```
f.readline()
f.readlines()
```

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Acceso a ficheros de Texto y Binarios

Método	Descripción
close()	Cierra un fichero, no tiene efecto si ya lo está
flush()	Vacía el buffer subyacente del flujo
read(n)	Lee al menos n caracteres del fichero. En caso de **None** o negativo lee hasta el final del fichero.
readline(n=-1)	Lee una línea del fichero leyendo al menos n bytes.
readlines(n=-1)	Lee un conjunto de líneas del fichero.
seek(offset,from=SEEK_SET)	Establece la nueva posición del puntero de lectura – escritura a partir de un punto.
tell()	Devuelve la posición del puntero de lectura.
write(s)	Escribe la cadena en el fichero.
writelines(lines)	Escribe un conjunto de líneas en el fichero

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Serialización

La serialización es el proceso de codificación de un objeto en un medio de almacenamiento (como puede ser un archivo, o un buffer de memoria), con el fin de transmitirlo a través de una conexión en red o almacenarlo en un fichero, como una serie de bytes o en un formato humanamente más legible como XML o JSON, entre otros. La serie de bytes o el formato pueden ser usados para crear un nuevo objeto que es idéntico en todo al original, incluido su estado interno (por tanto, el nuevo objeto es un clon del original).

El módulo **pickle** implementa un algoritmo para convertir un objeto arbitrario Python en una serie de bytes. El flujo de bytes que representa al objeto puede ser transmitido o almacenado y luego reconstruido para crear un nuevo objeto con las mismas características.

```
import pickle
import pprint

data1 = [{'a': 'A', 'b': 2, 'c': 3.0}]
print('BEFORE: ', end=' ')
pprint.pprint(data1)

data1_string = pickle.dumps(data1)

# data1_string puede ser transmitido o almacenado en disco

data2 = pickle.loads(data1_string)
print('AFTER : ', end=' ')
pprint.pprint(data2)

print('SAME? :', (data1 is data2))
print('EQUAL?:', (data1 == data2))
```

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Serialización

No todos los objetos pueden ser serializados: Sockets, gestores de archivos, conexiones de bases de datos, y otros objetos con estado de ejecución que dependen del sistema operativo, u otro proceso puede no ser posible guardalo. Los objetos que tienen atributos no serializables pueden definir **__getstate__()** y **__setstate__()** para devolver un subconjunto del estado de la instancia a ser serializada.

El protocolo pickle maneja automáticamente referencias circulares entre objetos, por lo que las estructuras de datos complejas no necesitan ningún manejo especial.

Utilización de los sistemas de ficheros

Al igual que podemos gestionar el contenido de ficheros, podemos gestionar los ficheros desde el nivel del sistema operativo. A este nivel podremos crear directorios, mover ficheros, etc. Veamos un ejemplo de las posibilidades que existen en la librearía **os, pathlib** y la librería **shutil**.

- import os. Uso de la librería (No se recomienda su uso) o `os.chdir(path), cambio de path actual.`
  - `os.getcwd(), path actual.`
  - `os.path, librería para manipulación de rutas a bajo nivel.`
  - os.listdir(path='.'). Retorna la lista de ficheros y directorios de una ruta.
  - `os.mkdir(path, mode=0o777, *, dir_fd=None). Crear un directorio.`
  - os.rmdir(path, *, dir_fd=None). Borra un directorio vacío.
  - os.remove(path, *, dir_fd=None). Borrar un fichero.
  - os.rename(src, dst, *, src_dir_fd=None, dst_dir_fd=None). Renombrar.

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Utilización de los sistemas de ficheros

- Import pathlib, librería para manipulación de rutas a nivel de objetos independientemente del SSOO.

https://docs.python.org/3/library/pathlib.html

- import shutil. Uso de la librearía ○ shutil.copyfile(src, dst, *, follow_symlinks=True), copiar un fichero usando descriptores.
  - o shutil.copy(src, dst, *, follow_symlinks=True), copiar un fichero usando rutas.
  - o shutil.copytree(src, dst, symlinks=False, ignore=None, copy_function=copy2,
  - o ignore_dangling_symlinks=False). Copia recursivamente un directorio entero.
  - o shutil.rmtree(path, ignore_errors=False, onerror=None). Borra un directorio entero.
  - o shutil.move(src, dst, copy_function=copy2). Mueve un fichero o directorio a otro sitio.

Tenemos que tener mucho cuidado con el uso de estas instrucciones ya que dependen del sistema operativo en las que las ejecutemos, por ejemplo, los parámetros **mode** de algunas instrucciones solo se utilizarán bajo Linux, no sobre Windows.

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Ejemplos

Desarrolla

- Dado un fichero con una palabra por línea, crear un fichero de salida con las palabras ordenadas.
- Dados dos ficheros con una palabra por línea, crear un fichero de salida que sea la concatenación de las correspondientes líneas de cada fichero.
- Realiza la aplicación code_0009_0 (Listas que hacer TODO almacenadas)

```
# ordena_palabras.py

def ordenar_palabras(entrada, salida):
    with open(entrada, 'r', encoding='utf-8') as f:
        palabras = [line.strip() for line in f.readlines() if line.strip()]

    palabras.sort()

    with open(salida, 'w', encoding='utf-8') as f:
        for palabra in palabras:
            f.write(palabra + '\n')

# Ejemplo de uso
ordenar_palabras('palabras.txt', 'palabras_ordenadas.txt')
```

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Ejemplos

1º Ejercicio

Escribir en un fichero llamado secuencias. las siguientes secuencias:

```
ACTG
GATA
```

2º Ejercicio

Escribir los números del 1 al 100 en un fichero.

3º Ejercicio

Escribir en un fichero separado por tabuladores la información de un paciente que tenemos disponible en el siguiente diccionario:

```
paciente = {'nombre': 'Daniel', 'edad': 42, 'Diabetico': True}
```

4º Ejercicio

Dado el siguiente fichero con información sobre pacientes:

```
Nombre edad Diabetico
Daniel 42 Si
Jose 15 Si
Manolo 50 No
Alicia 12 No
```

Imprimir en un nuevo fichero los pacientes que tienen más de 20 años y no son diabéticos.

5º Ejercicio

Ejercicio - Máximo y mínimo

Implementa un programa que muestre por pantalla los valores máximos y mínimos del archivo ‘numeros.txt’.

GESTIÓN DE DATOS EN PYTHON

Gestión de ficheros

Ejemplos

Ejercicio - Notas de alumnos

El archivo ‘alumnos_notas.txt’ contiene una lista de 10 alumnos y las notas que han obtenido en cada asignatura. El número de asignaturas de cada alumno es variable. Implementa un programa que muestre por pantalla la nota media de cada alumno junto a su nombre y apellido, ordenado por nota media de mayor a menor.

Ejercicio - Ordenando archivos

Implementa un programa que pida al usuario un nombre de archivo A para lectura y otro nombre de archivo B para escritura. Leerá el contenido del archivo A (por ejemplo ‘usa_personas.txt’) y lo escribirá ordenado alfabéticamente en B (por ejemplo ‘usa_personas_sorted.txt’).

Ejercicio - Nombre y apellidos

Implementa un programa que genere aleatoriamente nombres de persona (combinando nombres y apellidos de ‘usa_nombres.txt’ y ‘usa_apellidos.txt’). Se le pedirá al usuario cuántos nombres de persona desea generar y a qué archivo **añadirlos** (por ejemplo ‘usa_personas_generado.txt’).

Ejercicio - Búsqueda en PI

Implementa un programa que pida al usuario un número de cualquier longitud, como por ejemplo “1234”, y le diga al usuario si dicho número aparece en el primer millón de decimales del número pi (están en el archivo ‘pi-million.txt’). No está permitido utilizar ninguna librería ni clase ni método que realice la búsqueda. Debes implementar el algoritmo de búsqueda tú.

GESTIÓN DE DATOS EN PYTHON

XML

XML es el acrónimo de Extensible Markup Language, es decir, es un lenguaje de marcado que define un conjunto de reglas para la codificación de documentos. El lenguaje de marcado es un conjunto de códigos que se pueden aplicar en el análisis de datos o la lectura de textos creados por computadoras o personas. El lenguaje XML proporciona una plataforma para definir elementos para crear un formato y generar un lenguaje personalizado.

Un archivo XML se divide en dos partes: DTD y DATA. La parte DTD consiste en metadatos administrativos, como declaración XML, instrucción de procesamiento opcional, declaración de tipo de documento y comentarios. La parte del DATA se compone de dos partes: estructural y de contenido (presente en los textos simples).

El diseño XML se centra en la simplicidad, la generalidad y la facilidad de uso y, por lo tanto, se utiliza para varios servicios web. Tanto es así que hay sistemas destinados a ayudar en la definición de lenguajes basados en XML, así como APIs que ayudan en el procesamiento de datos XML – que no deben confundirse con HTML.

Las interfaces de Python para procesar XML están agrupadas en el paquete xml.

GESTIÓN DE DATOS EN PYTHON

XML. *Librerías Relacionadas con XML*

SAX

El paquete xml.sax provee un número de módulos que implementan la API Simple para la interfaz XML (SAX) para Python. Una aplicación SAX típica usa tres tipos de objetos: lectores, gestores y fuentes de entrada. «Lector» en este contexto es otro término para analizador, por ejemplo, alguna pieza de código que lee los bytes o caracteres de la fuente de entrada, y produce una secuencia de eventos. Los eventos luego se distribuyen a los objetos gestores, por ejemplo, el lector invoca un método en el gestor. Una aplicación SAX debe por tanto obtener un objeto lector, crear o abrir una fuente de entrada, crear los gestores, y conectar esos objetos juntos. Como paso final de preparación, el lector es llamado para analizar la entrada. Durante el análisis, los métodos en los objetos gestores son llamados basados en eventos estructurales y sintácticos de los datos introducidos.

https://docs.python.org/es/3/library/xml.sax.html

DOM

El Modelo de Objetos del Documento, o «DOM» por sus siglas en inglés, es un lenguaje API del Consorcio World Wide Web (W3C) para acceder y modificar documentos XML. Una implementación del DOM presenta los documentos XML como un árbol, o permite al código cliente construir dichas estructuras desde cero para luego darles acceso a la estructura a través de un conjunto de objetos que implementaron interfaces conocidas.

El DOM es extremadamente útil para aplicaciones de acceso directo. SAX sólo te permite la vista de una parte del documento a la vez. Si estás mirando un elemento SAX, no tienes acceso a otro. Si estás viendo un nodo de texto, no tienes acceso al elemento contendor. Cuando desarrollas una aplicación SAX, necesitas registrar la posición de tu programa en el documento en algún lado de tu código. SAX no lo hace por ti. Además, desafortunadamente no podrás mirar hacia adelante (look ahead) en el documento XML

Las aplicaciones DOM típicamente empiezan al diseccionar (parse) el XML en un DOM. Existe una clase objeto llamada DOMImplementation que da acceso a métodos de creación. Una vez que tengas un objeto del documento del DOM, puedes acceder a las partes de tu documento XML a través de sus propiedades y métodos. Estas propiedades están definidas en la especificación del DOM.

https://docs.python.org/es/3.9/library/xml.dom.html#module-xml.dom

GESTIÓN DE DATOS EN PYTHON

XML. *Creación de un documento XML*

```
<?xml version="1.0"?>
<genre catalogue="Pop">
   <song title="No Tears Left to Cry">
      <artist>
         Ariana Grande
      </artist>
      <year>2018</year>
      <album>Sweetener</album>
   </song>
   <song title="Delicate">
      <artist>
         Taylor Swift
      </artist>
      <year>2018</year>
      <album>Reputation</album>
   </song>
   <song title="Mrs. Potato Head">
      <artist>
         Melanie Martinez
      </artist>
      <year>2015</year>
      <album>Cry Baby</album>
   </song>
</genre>
```

SAX

```
import xml.sax
class SongHandler(xml.sax.ContentHandler):
   def __init__(self):
      self.CurrentData=''
      self.artist=''
      self.year=''
      self.album=''
   def startElement(self,tag,attributes):
      self.CurrentData=tag
      if tag=='song':
         print('Song:')
         title=attributes['title']
         print(f'Title: {title}')
   def endElement(self,tag):
      if self.CurrentData=='artist':
         print(f'Artist: {self.artist}')
      elif self.CurrentData=='year':
         print(f'Year: {self.year}')
      elif self.CurrentData=='album':
         print(f'Album: {self.album}')
      self.CurrentData=''
   def Characters(self,content):
      if self.CurrentData=='artist':
         self.artist=content
      elif self.CurrentData=='year':
         self.year=content
      elif self.CurrentData=='album':
         self.album=content
if __name__=='__main__':
parser=xml.sax.make_parser() #creating an XMLReader
parser.setFeature(xml.sax.handler.feature_namespaces,0)
#turning off namespaces
Handler=SongHandler()
parser.setContentHandler(Handler)
#overriding default ContextHandler
parser.parse('songs.xml')
```

GESTIÓN DE DATOS EN PYTHON

XML. *Creación de un documento XML*

```
<?xml version="1.0"?>
<genre catalogue="Pop">
   <song title="No Tears Left to Cry">
      <artist>
         Ariana Grande
      </artist>
      <year>2018</year>
      <album>Sweetener</album>
   </song>
   <song title="Delicate">
      <artist>
         Taylor Swift
      </artist>
      <year>2018</year>
      <album>Reputation</album>
   </song>
   <song title="Mrs. Potato Head">
      <artist>
         Melanie Martinez
      </artist>
      <year>2015</year>
      <album>Cry Baby</album>
   </song>
</genre>
```

DOM

```
from xml.dom.minidom import parse
import xml.dom.minidom
import os

os.chdir('C:\\Users\\lifei\\Desktop')
DOMTree = xml.dom.minidom.parse("songs.xml") #Opening the XML document
genre=DOMTree.documentElement

if genre.hasAttribute('catalogue'):
    print(f'Root: {genre.getAttribute("catalogue")}')

songs=genre.getElementsByTagName('song') #Get all songs in the
for song in songs: #Print each song's details
    print('Song:')
    if song.hasAttribute('title'):
        print(f'Title: {song.getAttribute("title")}')
    artist=song.getElementsByTagName('artist')[0]
    print(f'Artist: {artist.firstChild.data}')
    year=song.getElementsByTagName('year')[0]
    print(f'Release Year: {year.firstChild.data}')
    album=song.getElementsByTagName('album')[0]
    print(f'Album: {album.firstChild.data}')
```

GESTIÓN DE DATOS EN PYTHON

JSON.

¿Qué es Json?

JSON (JavaScript Object Notation) es un **formato de intercambio de datos** ampliamente utilizado por su sencillez y legibilidad. Permite representar estructuras de datos complejas (como diccionarios y listas) de forma compacta y fácil de entender tanto por humanos como por máquinas.

Comparación con estructuras de Python

JSON	Python
{"clave": "valor"}	{"clave": "valor"} (diccionario)
[1, 2, 3]	[1, 2, 3] (lista)
"texto"	"texto" (str)
true	True (bool)
null	None

Ejemplo visual:

Un ejemplo de objeto JSON que incluye distintos tipos de datos:

```
{
  "nombre": "Laura",
  "edad": 28,
  "activo": true,
  "hobbies": ["leer", "pintar"],
  "direccion": {
    "ciudad": "Madrid",
    "codigo_postal": 28001
  }
}
```

GESTIÓN DE DATOS EN PYTHON

JSON.

Cargar JSON en Python

Desde un string

Cuando los datos JSON están contenidos en un string (por ejemplo, si vienen de una API o archivo de texto), usamos **json.loads()** para convertirlos a estructuras de Python.

```
import json
texto_json = '{"nombre": "Laura", "edad": 28}'
datos = json.loads(texto_json)
print(datos["nombre"])  # Laura
```

Desde un archivo

Cuando el JSON está guardado en un archivo, usamos **json.load()** para leerlo directamente y convertirlo en estructuras de Python:

```
with open("datos.json", "r") as archivo:
    datos = json.load(archivo)
```

GESTIÓN DE DATOS EN PYTHON

JSON.

Tipos de datos en JSON

Es importante conocer cómo se traducen los tipos JSON a sus equivalentes en Python. Esto permite recorrer o manipular correctamente los datos una vez cargados.

JSON	Python	Ejemplo
Objeto	dict	{"clave": "valor"}
Array	list	["a", "b", "c"]
Cadena	str	"texto"
Número	int / float	42, 3.14
Booleano	bool	true, false → True, False
Nulo	None	null → None

GESTIÓN DE DATOS EN PYTHON

JSON.

Cómo Recorrer un Json

A. Recorrer un diccionario (clave-valor)

Una vez convertido el JSON a un diccionario de Python, podemos usar .items() para recorrer sus pares clave-valor:

```
for clave, valor in datos.items():
    print(f"{clave}: {valor}")
```

B. Recorrer una lista

Si una de las claves del JSON contiene una lista (por ejemplo, una lista de aficiones o direcciones), podemos recorrerla con un for tradicional:

```
for hobby in datos["hobbies"]:
    print(hobby)
```

C. Acceder a un diccionario anidado

En muchos casos, el JSON tiene estructuras anidadas (diccionarios dentro de otros). Para acceder a ellas, usamos el nombre de la clave y luego .items():

```
for campo, valor in datos["direccion"].items():
    print(f"{campo}: {valor}")
```

GESTIÓN DE DATOS EN PYTHON

JSON.

Cómo Recorrer un Json

D. Recorrido completo (estructurado)

Este ejemplo general permite recorrer cualquier JSON, identificando si un valor es una lista, un diccionario o un dato simple, y actuando en consecuencia:

```
for clave, valor in datos.items():
    if isinstance(valor, dict):
        print(f"{clave}:")
        for subclave, subvalor in valor.items():
            print(f"  {subclave}: {subvalor}")
    elif isinstance(valor, list):
        print(f"{clave}:")
        for item in valor:
            print(f"  - {item}")
    else:
        print(f"{clave}: {valor}")
```

Ejemplo de json:

```
datos = {
    "nombre": "Laura",
    "edad": 28,
    "hobbies": ["leer", "pintar"],
    "direccion": {
        "ciudad": "Madrid",
        "codigo_postal": 28001
    }
}
```

Resultado del código:

```
nombre: Laura
edad: 28
hobbies:
  - leer
  - pintar
direccion:
  ciudad: Madrid
  codigo_postal: 28001
```

GESTIÓN DE DATOS EN PYTHON

JSON.

Cómo Crear o Modificar un Json desde Python

Crear: consiste en construir una estructura de datos en Python y guardarla por primera vez como un archivo .json.

Modificar: implica leer un archivo JSON existente, actualizar su contenido en memoria, y volver a guardarlo con los cambios.

Para ambas tareas usamos el módulo estándar json.

```
import json
```

Crear un archivo JSON desde cero

Caso 1: Diccionario plano

Creamos un diccionario con datos simples (strings, números, booleanos) y lo guardamos en un archivo JSON con json.dump().

```
persona = {
    "nombre": "Ana",
    "edad": 25,
    "activo": True
}
with open("persona.json", "w") as archivo:
    json.dump(persona, archivo, indent=4)
```

indent=4 sirve para que el archivo tenga formato legible.

GESTIÓN DE DATOS EN PYTHON

JSON.

Cómo Crear o Modificar un Json desde Python

Crear un archivo JSON desde cero

Caso 2: Diccionario con lista

Añadimos una lista dentro del diccionario. El proceso es el mismo.

```
persona = {
    "nombre": "Carlos",
    "edad": 30,
    "lenguajes": ["Python", "Java", "C++"]
}

with open("carlos.json", "w") as archivo:
    json.dump(persona, archivo, indent=4)
```

Caso 3: Diccionario con diccionario anidado

Creamos un objeto más complejo con información jerárquica (ej. dirección dentro de persona).

```
persona = {
    "nombre": "Lucía",
    "edad": 27,
    "direccion": {
        "calle": "Gran Vía",
        "ciudad": "Madrid",
        "cp": 28013
    }
}

with open("lucia.json", "w") as archivo:
    json.dump(persona, archivo, indent=4)
```

GESTIÓN DE DATOS EN PYTHON

JSON.

Cómo Crear o Modificar un Json desde Python

Crear un archivo JSON desde cero

Caso 4: Lista de diccionarios

A veces necesitamos guardar una colección de objetos, por ejemplo, una lista de usuarios.

```
usuarios = [
    {"id": 1, "nombre": "Ana"},
    {"id": 2, "nombre": "Luis"},
    {"id": 3, "nombre": "Marta"}
]

with open("usuarios.json", "w") as archivo:
    json.dump(usuarios, archivo, indent=4)
```

Modificar un archivo JSON existente

Para modificarlo, hay que seguir estos tres pasos:

- Leer el archivo con json.load()
- Modificar la estructura en memoria
- Volver a guardar con json.dump()

GESTIÓN DE DATOS EN PYTHON

JSON.

Cómo Crear o Modificar un Json desde Python

Modificar un archivo JSON existente

Ejemplo: Añadir un nuevo usuario

```
# Paso 1: Leer el archivo
with open("usuarios.json", "r") as archivo:
    usuarios = json.load(archivo)

# Paso 2: Modificar los datos
usuarios.append({"id": 3, "nombre": "Elena"})

# Paso 3: Guardar el archivo actualizado
with open("usuarios.json", "w") as archivo:
    json.dump(usuarios, archivo, indent=4)
```

Archivo usuarios.json después de modificar:

```
[
    {
        "id": 1,
        "nombre": "Luis Miguel"
    },
    {
        "id": 3,
        "nombre": "Elena"
    }
]
```

Cambiar un valor existente

```
usuarios[0]["nombre"] = "Luis Miguel"
```

Eliminar un elemento

```
usuarios.pop(1)  # Elimina el segundo usuario
```

GESTIÓN DE DATOS EN PYTHON

JSON.

Cómo Crear o Modificar un Json desde Python

Ejercicio práctico: Leer, modificar y guardar un archivo JSON

Supón que tienes un archivo llamado alumnos.json con la siguiente información:

```
[
    {"nombre": "Lucía", "edad": 20},
    {"nombre": "Javier", "edad": 21},
    {"nombre": "Elena", "edad": 19}
]
```

Realiza los siguientes pasos desde Python:

- Carga el archivo alumnos.json.
- Añade un nuevo alumno llamado Daniel, de 22 años.
- Cambia la edad de Lucía a 21.
- Elimina el alumno llamado Javier.
- Guarda el archivo actualizado.
- Muestra por pantalla el contenido final.